balanced tree will mean that `h = log(n)` and in turn that all these methods will run in logarithmic time.

Unfortunately, a k-d tree is not a self-balancing tree, like RB-trees or 2-3-trees, to name a few: this means that, if we perform a great number of insertions and deletions on a tree, on average the tree will be tendentially balanced, but we have no guarantees on that. Moreover, if we resolve ties on coordinate comparison by always going to the same side, then it is proven that we will break the balance, over many operations.

To solve this problem, we can slightly change the compare method, as we had defined it in listing 9.2, so that it will never return 0: whenever we have a match, half of the times it will return -1, and half of the times +1, therefore achieving a better balance for the tree. The choice needs to be consistent, so we can't use randomness and have a perfect balance; instead, a possible solution to implement this correction, as shown in listing 9.5, is to return -1 when the node's level is even, and +1 when it's odd (or vice versa, doesn't really matter, as long as it's consistent).

Listing 9.5 Revised compare

```
function compare(point, node)                                    #1
  s ← sign(getPointKey(point, node.level) - getNodeKey(node))     #2
  if s == 0 then                                                 #3
    return node.level % 2 == 0 ? -1 : +1
  else
    return s
```

#1 The signature of the method remains unchanged.
#2 We store the value of the sign of the difference between the components, i.e. what was computed by the old